



OS-Project2

주제

kernel thread in xv6

작성자

전예원 / 컴퓨터소프트웨어학부 / 2023096608 / 3학년 / 12913

목차

1. Code analysis
2. Design
3. Implementation
4. Results
5. Troubleshooting

1. Code Analysis

proc.h

```
struct mm_struct {
    pagetable_t pagetable;    // page table root
    uint64 sz;                // user memory size (bytes)
    int refcount;             // number of tasks sharing this mm
    struct spinlock lock;
};

#define CLONE_VM      0x0100    // share address space (thread)

// Per-process (per-task) state.
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;      // Process state
    void *chan;                // If non-zero, sleeping on chan
    int killed;                // If non-zero, have been killed
    int xstate;                // Exit status to be returned to parent's wait
};
```

```

int pid; // Process / thread id (Linux's "TID")

// wait_lock must be held when using this:
struct proc *parent; // Parent process

// these are private to the process, so p->lock need not be held.
uint64 kstack; // Virtual address of kernel stack
struct mm_struct *mm; // Shared address space (replaces pagetable
+sz)
struct trapframe *trapframe; // data page for trampoline.S
struct context context; // swtch() here to run process
struct file *ofile[NOFILE]; // Open files
struct inode *cwd; // Current directory
char name[16]; // Process name (debugging)

// === thread group ===
struct proc *group_leader; // group leader (self if not a thread)
int tgid; // thread group id (== leader->pid)
uint64 tf_va; // user VA where p->trapframe is mapped
void *ustack; // user stack base (only set for threads)
};

```

기존의 xv6에는 struct proc안에 실행 상태, kernel stack, context, page table, memory size, file table 등을 모두 한 구조체에 묶어두었다.

- 기존 구조에서 새로운 execution unit을 만드려면 새 address space도 같이 생긴다.
- 이는 같은 주소 공간을 공유하는 thread 입장에서 적절하지 않다.
- 따라서 thread를 만들기 위해 먼저 실행 단위(CPU scheduling 단위)는 proc안에, memory management를 mm_struct를 통해 분리하여 여러 thread가 같은 주소 공간을 가리킬 수 있게 선언하였다.

mm_struct 분석

- `pagetable` 은 해당 address space의 page table이다.
- `sz` 는 process memory size로, `sbrk()` 등으로 증가하는 사용자 메모리 크기를 의미한다.
- `refcount` 는 이 `mm_struct` 를 공유하고 있는 task/thread 개수다.
- `lock` 은 여러 thread가 같은 `mm_struct` 를 공유하므로, refcount나 sz 등을 안전하게 수정하기 위한 spinlock이다.

→ process와 thread는 직접 page table을 갖는 것이 아닌, mm_struct를 가리킨다.

refcount 설정

The last one out cleans up

- 같은 address space를 공유하는 thread가 여러 개 있다면, 마지막 thread의 개수가 사라질 때만 page table과 mm_struct를 free해야한다.

```
mm_alloc() → ref_count = 1
mm_get() → ref_count++
mm_put() → ref_count--
if(ref_count == 0) free pagetable + free mm
```

- 중요한 규칙: 한번 mm_get()했으면 mm_put()도 정확히 한번 대응되어야한다.
 - 이 부분이 어긋날 시 memory leak이나 double free가 생길 수도 있다.

struct proc의 추가 field

- `group_leader` 는 thread group의 대표 task로, 일반 process로 시작한 첫번째 task가 leader가 된다고 볼 수 있다.
- `tgid` 는 thread group id로, leader의 pid와 같게 설정된다.
- `tf_va` 는 해당 thread의 trapframe이 user page table에서 매핑되는 virtual address이다.
- `ustack` 은 thread의 user stack base로, join시 이용된다.

→ 4개의 field는 process/thread마다 가지는 자원이다.

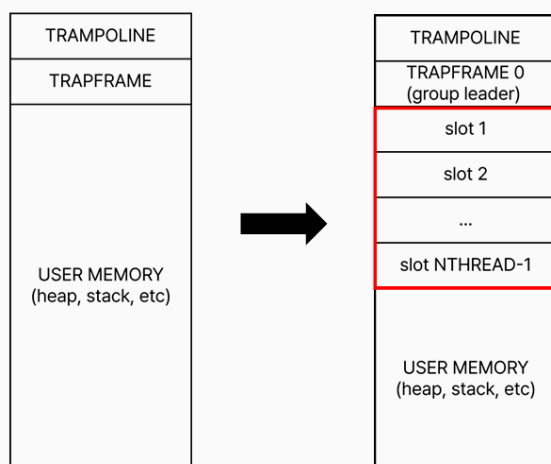
clone() system call

```
fork() == clone(SIGCHLD)
pthread_create() == clone(CLONE_VM | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD ...)
```

CLONE_VM이란 flag가 clone을 통해 들어오면 address space를 복사하지 않고 공유하게 하면 된다. 즉, thread처럼 동작하는 것이다.

- clone(0)이면 `fork`, clone(CLONE_VM)이면 `thread_create` 를 호출해주면 된다

One page table, Many trapframes



thread는 같은 page table을 공유하지만, trapframe은 각각 가지고 있어야함

- 이는 trapframe에 각 thread의 user register 상태가 저장되기 때문이다.

기존 xv6는 보통 하나의 process에 하나의 trapframe VA를 사용하나, 현재 thread가 여러개 생성될 수 있으므로 trapframe slot을 여러 개 두어 여러 개의 trapframe을 할당할 수 있도록 해주었다.

```
#define NTHREAD 8
#define TRAPFRAME (TRAMPOLINE - PGSIZE)
#define USERTOP (TRAPFRAME - NTHREAD * PGSIZE)
```

- `find_free_tf_va()` 를 이용하면, 아직 사용되지 않은 비어있는 trapframe VA slot을 찾아주면 된다.
 - 이 빈 slot을 찾아서 해당 thread의 trapframe을 매핑시켜주면 된다.

How threads find their own trapframe

thread마다 trapframe VA가 다르므로 trap 발생 시, kernel이 현재 thread의 trapframe이 어디 있는지 알기 위해 sscratch CSR을 사용한다.

- kernel에서 user로 돌아가기 직전, `sscratch = p - tf_va`로 설정한다.
- 다음 user mode에서 trap발생하여 kernel로 들어오면 trampoline의 `uservec`에서 해당 명령어를 실행하여 `a0`와 `sscratch` 값을 swap해준다.

```
csrrw a0, sscratch, a0
```

- 결과는 다음과 같다.

```
a0 = 현재 thread의 TRAPFRAME VA
sscratch = user mode 때의 원래 a0 값
```

New System call

clone()

```
int clone(void (*fn)(void *), void *arg, void *stack, int n_pages, int flags);
```

- `fn` 은 새 thread가 시작할 함수이다.
- `arg` 는 함수에 넘길 인자, 여러 값을 넘기고 싶으면 struct로 묶어서 pointer를 넘긴다.
- `stack` 은 새 thread가 사용할 user stack이다.
- `n_pages` 는 stack 크기를 page 단위로 나타낸다. kernel에서 새로운 thread의 `sp`에 설정하기 위해 사용
- `flags` 는 process처럼 만들지 thread처럼 만들지 결정한다.
 - `CLONE_VM` 이면 thread
 - 0이면 process

→ `proc.c`의 `kclone` 에서 `CLONE_VM`을 통한 thread 부분을 수정하면 된다.

`clone`이라는 kernel system call interface를 보면 stack을 인자로 받으므로, `int thread_create(void (*fn)(void *), void *arg, int n_pages);` 라는 user API에서 stack을 `malloc(n_pages * PGSIZE)`로 할당하고 그 pointer를 `clone()`에 넘겨야한다.

join()

```
int join(void **stack);
```

해당 system call을 통해 종료된 thread의 stack 주소를 `int thread_join(void);` 가 받아서 `free()` 해주면 된다.

2. Design

먼저 thread와 process를 구분하여 설계해주는 것이 가장 큰 특징이다.

- 새로운 process가 생성되면 독립적인 mm_struct를 할당하고 reference count를 1로 초기화한다. 반면 thread가 생성될 경우에는 새로운 address space를 만들지 않고 leader가 사용 중인 mm_struct를 공유하며, 이때 reference count를 증가시키도록 설계하는게 맞다고 생각했다.
- 각 task가 종료되어 더 이상 해당 address space를 사용하지 않게 되면 reference count를 감소시킨다. reference count가 0이 되는 경우에만 page table과 mm_struct를 실제로 해제하면 된다.
- join은 일반 process의 wait와 분리되어야 한다. join은 같은 thread group에 속한 task만 대상으로 탐색하며, 종료된 thread가 발견되면 해당 thread의 user stack 주소를 반환하고 resource를 회수하도록 하면 된다고 보았다.
- 일반 thread가 exit을 호출하는 경우에는 해당 thread 하나만 종료되어야 하며, 같은 group의 leader나 sibling thread들은 계속 실행될 수 있어야 한다. 따라서 non-leader thread는 자신의 resource를 정리한 뒤 ZOMBIE 상태가 되고, leader가 이후 join()을 통해 이를 회수하도록 하면 된다. leader는 sibling thread를 모두 회수한 뒤 종료되도록 설계하였다.
- Thread가 존재할 경우, kill(pid)가 하나의 task만 종료시키면, 같은 process에 속한 다른 thread들은 계속 실행될 수 있다. kill을 pid가 leader의 pid인지 thread의 pid인지와 관계없이 해당 task가 속한 thread group 전체를 종료 대상으로 삼도록 설계하면 된다. 따라서 tgid를 이용하면 된다고 판단하였다.
- kexec는 먼저 실행할 program 로딩에 성공하여 user stack 등이 구성 완료된 경우 sibling들을 회수한 뒤, page table을 new page table로 교체해주고 old page table을 해제해주는 방향이 맞다고 생각하였다.

3. Implementation

step1

mm_get

```
void
mm_get(struct mm_struct *mm)
{
    acquire(&mm->lock);
    mm->refcount++;
    require(&mm->lock);
}
```

- 어떤 task가 기존 mm_struct를 새로 공유하기 시작할 때 호출한다.
- refcount++해주면 되는데, 여러 task가 동시 접근할 수도 있으므로 lock을 걸어준다.

mm_put

```

void
mm_put(struct mm_struct *mm)
{
    //마지막 task가 없다면 mm_struct 해제
    int isLast = 0;
    acquire(&mm->lock);
    mm->refcount--;
    if(mm->refcount == 0) isLast = 1;
    release(&mm->lock);

    if(isLast){
        proc_freepagetable(mm->pagetable, mm->sz);
        kfree(mm);
    }
}

```

- 역시나 refcount는 여러 task 동시 접근이 가능하므로 lock을 잡고 refcount—해준다.
- 마지막 task라면 page table을 해제한 뒤 mm을 free하면 된다.
- page table 해제는 lock 밖에서 해도 괜찮다. 마지막 참조자인 것을 확인한 뒤라 다른 task가 그 mm을 쓰면 잘못된 경우이다.

find_tree_tf_va

```

uint64
find_free_tf_va(pagetable_t pagetable)
{
    //비어있는 trapframe VA 찾기 없으면 0
    for(int i=0; i<NTHREAD; i++){
        uint64 va = TRAPFRAME - i * PGSIZE;
        if(!ismapped(pagetable, va)) return va;
    }
    return 0; //없다는 뜻
}

```

- thread의 trapframe slot을 찾기 위해서는 TRAPFRAME-PGSIZE, TRAPFRAME-2*PGSIZE로 순차확인 하면 되며, 총 slot의 개수인 NTHREAD만큼 순회하면 된다.
- slot을 찾아서 사용중인 slot인지 확인하는 방법은 kernel/vm.c에 있는 ismapped()를 통해 확인할 수 있다.
- 비어있는 걸 찾는 즉시 해당 slot의 virtual address를 return하고, 없으면 0을 return해주면 된다.
- slot 0인 TRAPFRAME은 group leader가 이미 점유하는 상태이며, 현재 proc_pagetable()이 leader의 trapframe을 TRAPFRAME에 map하므로 skip된다고 볼 수 있다.

freeproc()

```

static void
freeproc(struct proc *p)
{

```

```

//leader가 아닌 thread의 trapframe slot mapping 제거
if(p->mm && p->tf_va && p->tf_va != TRAPFRAME)
    acquire(&p->mm->lock);
    uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);
    release(&p->mm->lock);

if (p->trapframe) {
    kfree((void *)p->trapframe);
    p->trapframe = 0;
}
p->tf_va = 0;

if (p->mm) {
    mm_put(p->mm);
    p->mm = 0;
}

p->pid = 0;
p->parent = 0;
p->name[0] = 0;
p->chan = 0;
p->killed = 0;
p->xstate = 0;
p->state = UNUSED;
p->group_leader = 0;
p->tgid = 0;
p->ustack = 0;
}

```

- thread가 생기면 p→trapframe를 free하기 전에, shared page table에 있는 tf_va mapping도 지워야한다.
 - p→mm→pagetable은 thread들이 공유하는데, 죽은 thread의 trapframe mapping이 남으면 slot 재사용이 안되기 때문이다.
- free 순서는 p→mm과 p→tf_va가 있으면 unmap 해준 뒤, p→trapframe free해주고 mm_put()을 해주면 된다.
 - uvmunmap에서 마지막 인자인 do_free를 0으로 넣어주어야한다. 1이면 trapframe까지 free하기 때문에 중복 free가 된다.
- leader의 trapframe을 먼저 unmap하면 proc_freepagetable()에서 다시 uvmunmap을 호출하여 중복 호출하는 경우가 생긴다. 이를 freeproc()에서는 thread slot만 unmap하고 leader slot은 마지막 mm_put()의 proc_freepagetable()에 하도록 결정하여 구현하였다.
- kclone시 동시에 page table 수정하는 것을 막기 위해 uvmunmap을 lock 걸어야 한다.

step2

kclone()

```

int
kclone(uint64 fn, uint64 arg, uint64 stack, int n_pages, int flags)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    if ((np = allocproc()) == 0)
        return -1;

    if (flags & CLONE_VM) {
        // === thread path ===
        uint64 stack_top;
        uint64 current_tf_va;

        //인자 확인
        if(stack == 0 || n_pages <= 0) goto fail;

        stack_top = stack + (uint64)n_pages*PGSIZE;

        //thread이므로 주소 공간 공유해야함
        np->mm = p->mm;
        mm_get(np->mm); //refcount++

        //trapframe을 page table에 매핑
        acquire(&np->mm->lock);

        current_tf_va = find_free_tf_va(np->mm->pagetable);
        if(current_tf_va == 0){ //먼저 page table에 빈 slot 찾아야함
            release(&np->mm->lock);
            goto fail;
        }

        if(mappages(np->mm->pagetable, current_tf_va, PGSIZE, (uint64)np->trap
frame, PTE_R|PTE_W) < 0){
            release(&np->mm->lock);
            goto fail;
        } //매핑 완료

        np->tf_va = current_tf_va;
        release(&np->mm->lock);

        //thread 정보 update
        np->group_leader = p->group_leader;
        np->tgid = p->tgid;
        np->ustack = (void *)stack;
    }
}

```



```

//실행
*(np->trapframe) = *(p->trapframe);
np->trapframe->epc = fn;
np->trapframe->sp = stack_top;
np->trapframe->a0 = arg;
np->trapframe->ra = 0;

} else {
    // === fork path ===
    if ((np->mm = mm_alloc()) == 0)
        goto fail;
    if ((np->mm->pagetable = proc_pagetable(np)) == 0)
        goto fail;
    if (uvmcopy(p->mm->pagetable, np->mm->pagetable, p->mm->sz) < 0)
        goto fail;
    np->mm->sz = p->mm->sz;
    np->group_leader = np;
    np->tgid          = np->pid;

    // copy parent's user registers; child resumes after the syscall.
    *(np->trapframe) = *(p->trapframe);
    np->trapframe->a0 = 0;    // child's clone() returns 0
}

// === common: files, cwd, name ===
for (i = 0; i < NOFILE; i++)
    if (p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
np->cwd = idup(p->cwd);

safestrcpy(np->name, p->name, sizeof(p->name));

pid = np->pid;

release(&np->lock);

acquire(&wait_lock);
np->parent = p;
release(&wait_lock);

acquire(&np->lock);
np->state = RUNNABLE;
release(&np->lock);

return pid;

fail:
    freeproc(np);

```

```

    release(&np->lock);
    return -1;
}

```

kclone은 flags에 따라 thread를 만들지, process를 만들지 결정하고, 두 종류의 task를 만드는 함수이다.

- 먼저 allocproc()를 하면, thread 또는 process 둘다 pid, trapframe, context, kstack 모두 준비되어있다.
 - 즉 물리 trapframe을 새로 할당하지 않고 이미 할당된 np→trapframe을 page table에 매핑만 해주면 된다.
- 먼저 stack과 pages의 인자가 올바른지 확인해주었다.
- 이후 초기 stack pointer를 stack 주소에 page개수와 PGSIZE를 곱한 뒤 더하여 stack_top을 먼저 결정해주었다.
- 그리고 부모와 address space를 공유해야하고 mm_get을 호출하여 refcount++를 해준다. 즉, 부모가 가진 mm_struct를 그대로 공유해주면 된다.
- 이후 page table안에 trapframe VA를 받기 위해 lock을 잡고, find_tree_tf_va()를 호출하여 받는다.
- 빈 slot이 있어 할당 받으면, mappages()를 통해 page table에 mapping 해준다.
 - 이때 PTE_U가 없는 이유는 trapframe이 사용자 프로그램이 직접 읽고 쓰는 영역이 아니기 때문이다.
- 이때, find_tree_tf_va와 mappages는 같은 lock안에 묶여서 호출되어야한다. 중간에 lock이 풀리면 slot 상태가 달라질 수 있기 때문이다.
- 매핑 성공 시 np의 tf_va를 매핑받은 current_tf_va로 설정해주면 된다.
- thread group의 정보를 thread에도 설정해주면 된다. (group_leader, tgid, ustack)
 - 이때 ustack은 thread_create에서 할당받은 stack으로 설정해주면 된다.
- 마지막으로 새 thread의 사용자 실행상태를 설정해주면 된다.
 - 부모 trapframe을 복사해오고, thread 시작에 필요한 register(epc, sp, a0, ra)만 따로 설정해주면 된다.

thread_create()

```

int
thread_create(void (*fn)(void *), void *arg, int n_pages)
{
    if(n_pages <= 0) return -1; //잘못된 입력 거르기

    void* stack = malloc(n_pages*PGSIZE);
    if(stack == 0) return -1;

    int pid = clone(fn, arg, stack, n_pages, CLONE_VM); //thread clone
    if(pid < 0){ //clone 안되면 할당 해제
        free(stack); return -1;
    }
    return pid;
}

```

thread를 생성하며, 편하게 clone 호출을 위한 user-space wrapper 함수이다.

- 먼저, n_pages 인자가 제대로 전달되었는지 확인한다.
- 이후 새 thread가 사용할 stack을 할당한다.
- 이후 할당한 stack base 주소를 kclone으로 넘기면 kclone은 초기 sp를 계산하여 stack_top을 결정한다.
- 이후 thread마다 다른 pid를 가지므로 해당 pid를 반환해주면 된다.

kjoin()

```
int
kjoin(uint64 stack_addr)
{
    struct proc *pp;
    struct proc *p = myproc();
    int threadNum, pid;

    if(p != p->group_leader) return -1; //thread를 가진 주체가 join하기

    acquire(&wait_lock);

    for(;;){
        threadNum = 0;
        for(pp=proc; pp<&proc[NPROC]; pp++){
            if(pp == p || pp->group_leader != p) continue; //대상 아닌 thread 제외

            acquire(&pp->lock);
            threadNum++; //대상 thread 찾을

            if(pp->state == ZOMBIE){ //zombie면 상태 복사 후 freeproc 필수
                pid = pp->pid;

                if(stack_addr != 0 && copyout(p->mm->pagetable, stack_addr, (char
                *)&pp->ustack, sizeof(pp->ustack))<0){
                    release(&pp->lock);
                    release(&wait_lock);
                    return -1;
                }

                freeproc(pp);
                release(&pp->lock);
                release(&wait_lock);
                return pid;
            }
            release(&pp->lock);
        }
        if(!threadNum || killed(p)){ //기다릴 대상 없음
            release(&wait_lock);
            return -1;
        }
    }
}
```

```

    sleep(p, &wait_lock);
}
}

```

kjoin()함수는 종료된 sibling thread하나를 찾아 자원을 free한 뒤, 그 thread가 쓰던 user stack의 base 주소를 return하는 함수이다. kwait와 구조가 거의 같지만, 같은 thread group에 속하는 thread를 회수한다는 점만 다르다.

- stack_addr은 user space에 결과 stack pointer를 써줄 주소라고 볼 수 있다.
- 현재 p가 group_leader가 아니면, join하지 못하도록 먼저 필터링 해주었다.
- wait_lock을 잡아 thread가 영원히 sleep하지 않도록 방지해주었다.
- process 그룹을 순회하며 pp가 현재 실행 중인 p가 아니면서 group_leader가 p인, 즉 같은 thread group에 있는 thread 하나를 선택해서 해당 thread의 상태를 확인한다.
 - 이때, thread의 state를 확인해야하므로 pp → lock을 걸어서 작업한다.
 - threadNum은 아직 종료되지 않아도 해당 thread group에 thread가 존재한다는 의미이다.
 - thread가 종료되면, 해당 thread에서 stack 주소를 copyout을 통해 꺼내준다.
 - 이때, stack의 base가 return된다고 볼 수 있는데, 처음에 thread 생성 시 malloc해서 반환된 주소 그대로를 return 해줘야 free가 가능하다.
 - 만약 thread 상태가 zombie가 아니면 계속 탐색하도록 두었다.
- 만약, 기다릴 대상이 없는 경우 -1을 return한다.
- 살아있는 thread가 있다면, sleep해주면 된다.

thread_join()

```

int
thread_join(void)
{
    void *stack;
    int pid = join(&stack);

    if(pid < 0) return -1;
    free(stack);
    return pid;
}

```

join()을 호출하고, kernel이 return한 종료된 thread의 user stack을 해제하는 wrapper이다.

- stack 변수를 선언하여, 종료된 thread의 stack base 주소를 받을 수 있도록 한다.
- 그리고 종료된 thread의 pid를 return 받는다.
- join 실패 시에는 -1을 return하고 stack을 free해서는 안된다.
 - kjoin이 완벽하게 proc를 회수해야 해당 thread의 stack을 사용하는 일이 없기 때문이다.
- 이후 join 성공 시 stack을 free한 뒤 pid를 return 해주면 된다.

step3

kill_sibling_threads()

```
//step3에서 여러번 사용되는 부분 helper로 묶기
void
kill_sibling_threads(struct proc* gleader){
    struct proc* i;

    acquire(&wait_lock);
    for(;;){
        int isRemain = 0;

        for(i=proc; i<&proc[NPROC]; i++){
            if(i == gleader || i->group_leader != gleader) continue;

            acquire(&i->lock);

            if(i->state == ZOMBIE){
                freeproc(i);
            }else{
                i->killed = 1;

                if(i->state == SLEEPING) i->state = RUNNABLE;
                isRemain = 1;
            }
            release(&i->lock);
        }
        if(!isRemain){
            break;
        }
        sleep(gleader, &wait_lock);
    }
    release(&wait_lock);
}
```

kernel/proc.c에 현재 leader를 제외한 sibling thread를 종료시키고 회수하는 helper함수를 만들었다. 해당 함수는 여러번 쓰일 것 같아서 helper로 따로 정의하였다.

- 현재 group의 leader를 인자로 받아, sibling을 for문을 통해 순회하며 검색한다.
- isRemain을 정의하여 모든 sibling이 ZOMBIE 상태가 되어 회수될 때까지 반복한다.
- 만약 아직 살아있는 상태라면 killed flag를 1로 바꾼 뒤, sleeping 상태이면 RUNNABLE 상태로 바꿔주면 된다.
- 실행 중인 thread를 바로 freeproc하지 않고 ZOMBIE인 경우에만 free하도록 주의하면 된다.

kexit()

```

void
kexit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
        panic("init exiting");

    // Close all open files.
    for(int fd = 0; fd < NOFILE; fd++){
        if(p->ofile[fd]){
            struct file *f = p->ofile[fd];
            fileclose(f);
            p->ofile[fd] = 0;
        }
    }

    begin_op();
    iput(p->cwd);
    end_op();
    p->cwd = 0;

    //p 작업 전 sibling 먼저 정리 필수!
    if(p->group_leader == p) kill_sibling_threads(p);

    acquire(&wait_lock);

    // Give any children to init.
    reparent(p);

    // Parent might be sleeping in wait().
    // And the group leader might be sleeping in join().
    wakeup(p->parent);
    if (p->group_leader != p)
        wakeup(p->group_leader);

    acquire(&p->lock);

    p->xstate = status;
    p->state = ZOMBIE;

    release(&wait_lock);

    // Jump into the scheduler, never to return.
    sched();
    panic("zombie exit");
}

```

현재 실행중인 task가 group_leader 인 경우 sibling을 먼저 정리한 후 ZOMBIE 상태가 되어야한다.

- 따라서 p→group_leader가 자기 자신인 경우, helper 함수를 호출하여 sibling을 모두 정리하도록 해주었다.
- 일반 thread가 exit하는 경우, leader를 깨우고 자신만 ZOMBIE가 되므로 기존 코드 그대로 놔두어도 된다.

kwait()

```
int
kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();

    acquire(&wait_lock);

    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if (pp->parent != p || pp->group_leader != pp)
                continue;

            // make sure the child isn't still in exit() or swtch().
            acquire(&pp->lock);

            havekids = 1;
            if(pp->state == ZOMBIE){
                // Found one.
                pid = pp->pid;
                if(addr != 0 && copyout(p->mm->pagetable, addr,
                                         (char *)&pp->xstate,
                                         sizeof(pp->xstate)) < 0) {
                    release(&pp->lock);
                    release(&wait_lock);
                    return -1;
                }
                freeproc(pp);
                release(&pp->lock);
                release(&wait_lock);
                return pid;
            }
            release(&pp->lock);
        }

        // No point waiting if we don't have any children.
        if(!havekids || killed(p)){
            release(&wait_lock);
            return -1;
        }
    }
}
```

```

    }

    // Wait for a child to exit.
    sleep(p, &wait_lock); //DOC: wait-sleep
}
}

```

kwait의 경우, pdf에 group leader인 child만 count하라고 되어있었다. 따라서 올바른 조건은 process table을 순회하면서, 해당 process의 parent가 현재 process이면서, 해당 process가 group_leader인 경우이다.

- thread의 경우에도 kclone에서 $np \rightarrow parent = p$;로 부모가 지정된다는 점을 기억해야한다.
- 이에 따라, $pp \rightarrow group_leader \neq pp$ 인 경우를 추가적으로 검사하여 group_leader가 아닌 경우를 제외시켜 주면 된다.
 - 이를 제외하지 않으면 thread를 자식 process로 잘못판단하여 free할 수도 있다.

kill()

```

int
kill(int pid)
{
    struct proc *p;
    int kill_gid = -1;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            kill_gid = p->tgid;
            release(&p->lock);
            break;
        }
        release(&p->lock);
    }

    if(kill_gid == -1) return -1; //찾는 pid가 없는 경우

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->tgid == kill_gid){
            p->killed = 1;

            if(p->state == SLEEPING) p->state = RUNNABLE;
        }
        release(&p->lock);
    }
    return 0;
}

```


기존의 kkill인 경우 찾는 pid를 가진 task하나만 kill하는 구조라서, entire group을 kill하는 구조로 변화시켜 주어야 한다. 이엔 두가지 케이스가 있다. 먼저, pid가 process인 경우, 그리고 group에 속하는 thread인 경우이다. 하지만, proc 구조체 안에 tgid가 있다. tgid는 group_leader인 경우 자기 자신의 pid를 가지고, thread인 경우 group_leader의 pid를 가지므로 2가지 case를 구분하지 않고 한번에 처리할 수 있었다.

- 먼저 kill_gid 변수를 설정하여 pid를 가진 task가 존재할 시 tgid를 담을 수 있도록 하였다.
- proc table을 순회하며 tgid를 kill_gid에 담아주고, break한다.
- kill_gid가 여전히 -1인지 check하여 존재하는 task인지 확인해준다.
- 그리고 proc table을 다시 한번 더 순회하여 해당 tgid를 가진 task를 모두 killed를 1로 setting하고, SLEEPING인 경우 RUNNABLE 상태로 바꾸어 scheduling 대상이 되게 해주면 된다.

kexec()

```
int
kexec(char *path, char **argv)
{
    ...
    if(p != p->group_leader) return -1;

    begin_op();

    // Open the executable file.
    ...
    // Read the ELF header.
    ...

    // Load program into memory.
    ...

    p = myproc();
    uint64 oldsz = p->mm->sz;

    // Allocate some pages at the next page boundary.
    // Make the first inaccessible as a stack guard.
    // Use the rest as the user stack.
    ...

    // Copy argument strings into new stack, remember their
    // addresses in ustack[].
    ...

    // push a copy of ustack[], the array of argv[] pointers.
    ...
    // a0 and a1 contain arguments to user main(argc, argv)
    // argc is returned via the system call return
    // value, which goes in a0.
    p->trapframe->a1 = sp;
```

```

// Save program name for debugging.
...

kill_sibling_threads(p);

// Commit to the user image.
acquire(&p->mm->lock);
oldpagetable = p->mm->pagetable;
oldsz = p->mm->sz;
p->mm->pagetable = pagetable;
p->mm->sz = sz;
release(&p->mm->lock);
p->trapframe->epc = elf.entry; // initial program counter = ulib.c:star
t()
p->trapframe->sp = sp; // initial stack pointer
proc_freepagetable(oldpagetable, oldsz);

return argc; // this ends up in a0, the first argument to main(argc, arg
v)

bad:
if(pagetable)
    proc_freepagetable(pagetable, sz);
if(ip){
    iunlockput(ip);
    end_op();
}
return -1;
}

```

kexec의 조건은 일반 thread에서 호출한 exec()는 금지하고, leader가 exec()할 때는 sibling thread를 모두 종료하고 회수한 뒤 page table을 교체해야하는 것이다. 해당 코드가 길어 수정이 필요한 부분만 나타내었다.

- 먼저 kexec를 실행하는 proc가 thread가 아닌지를 먼저 검사하여, thread인 경우 return -1 해주었다.
- 그리고 새로 시작할 program이 load가 완료되지 않으면, 기존 실행하던 program으로 가야하므로 이것보다 먼저 sibling을 회수하면 기존 thread들이 사라지므로 이를 주의해주었다.
- 따라서 user image commit 전에 kill_sibling_threads 함수를 호출하여 thread들을 모두 회수해주었다.
- 그리고 new page table로 교체해주면 되는데, 이때 oldsz에 lock을 건 후의 p → mm → sz를 update 해주어야한다.
 - lock을 걸기 전에는 새 program이 준비되는데, 이때 기존의 page table에 접근하여 thread가 생성될수도 있기 때문이다.
- 마지막으로 old page table을 free 해주어야한다.

4. Results

step1 result

```
$ usertests forktest
usertests starting
test forktest: PASS
PASS ALL TESTS
```

왼쪽 사진처럼, step1 코드 수정 후 make qemu CPUS=1
으로 실행 시 forktest가 통과했음을 확인하였다.

step2 result

```
$ threadtest
basic: PASS
slots: PASS
threadtest: PASS
```

직접 threadtest를 만들어 test를 진행해보았다.

- basic은 thread를 생성하여, address space를 공유하는지 확인하고 join을 확인하는 흐름이다.
- slots는 동시 thread 수 제한과 trapframe slot 재사용을 확인하는 코드였다.

→ 두 가지 모두 정상적으로 통과함을 확인하였다.

final result

먼저, usertests의 test에 대해서 모든 검사를 실시해보았다.

```
PASS ALL TESTS
$
```

이렇게 모든 test에 대해 먼저 통과한 것을 확인할 수 있었다.

그리고, testcase를 받아서 실행해보았다.

```
[TEST#1]
Thread 0 start
Thread 1 start
Thread 1 end
Thread 2 start
Thread 2 end
Thread 3 start
Thread 3 end
Thread 4 start
Thread 4 end
Thread 0 end
TEST#1 Passed

[TEST#2]
Thread 0 start, iter=0
Thread 0 end
Thread 1 start, iter=1000
Thread 1 end
Thread 2 start, iter=2000
Thread 2 end
Thread 3 start, iter=3000
Thread 3 end
Thread 4 start, iter=4000
Thread 4 end
TEST#2 Passed

[TEST#3]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Child of thread 0 start
Child of thread 1 start
Child of thread 2 start
Child of thread 3 start
Child of thread 4 start
Child of thread 0 end
Child of thread 1 end
Child of thread 2 end
Child of thread 3 end
Child of thread 4 end
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#3 Passed
```

```

[TEST#4]
Thread 0 sbrk: old break = 0x00000000000015000
Thread 0 sbrk: new break = 0x00000000000029010
Thread 1 size = 0x00000000000029010
Thread 2 size = 0x00000000000029010
Thread 3 size = 0x00000000000029010
Thread 4 size = 0x00000000000029010
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#4 Passed
[TEST#5]
Thread 0 start, pid 6695
Thread 1 start, pid 6695
Thread 2 start, pid 6695
Thread 3 start, pid 6695
Thread 4 start, pid 6695
TEST#5 Passed
[TEST#6]
leader exec passed
TEST#6 Passed

```

- test1은 thread_create와 thread_join이 정상동작하는지, thread와 leader가 같은 address space 공유하는지 확인하는 test이다.
- test2는 여러 thread가 같은 전역 배열을 공유하는지, 그리고 여러 thread 생성과 join이 반복적으로 가능한지 확인하는 test이다.
- test3은 thread 안에서 호출한 fork가 별도의 공간을 가진 process를 만드는지, thread가 자신이 만든 child process를 wait하는지, kwait 올바르게 처리하였는지 확인하는 test이다.
- test4는 malloc으로 확장한 heap을 다른 thread들이 볼 수 있는지, sbrk(0)이 동일한 경우 공유 주소 공간 크기를 반영하는지 등을 test한다.
- test5는 kkill과 kexit 그리고 kwait가 맞물려 올바르게 동작하는지 확인하는 test이다.
- test6는 exec이 조건대로 잘 동작하는지 확인하는 test이다. thread는 exec 불가이므로 이를 추가하였다.

5. Troubleshooting

Leader 종료 시 Sibling thread 회수 과정의 무한 대기 문제

Leader가 exit()을 호출할 때에는 sibling thread를 종료시키고 모두 회수한 뒤 자신이 종료되어야 한다.

처음에는 sibling thread가 존재하지만 기준으로 대기하도록 생각했지만, 마지막 sibling이 이미 ZOMBIE여서 회수된 경우에도 leader가 다시 sleep()할 수 있다는 문제가 있었다. 이 경우 더 이상 leader를 깨워 줄 sibling이 남아 있지 않으므로 thread group의 종료가 완료되지 않았다. 이 때문에 내가 만든 thread test의 test5와 test6이 fail이 났다.

이를 해결하기 위해 sibling을 순회하면서 ZOMBIE thread는 즉시 freeproc()으로 회수하고, 아직 종료되지 않은

thread가 있는 경우에만 대기하도록 하도록 조건문을 수정하였다.

이를 통해 leader 종료 및 exec() 수행 시 sibling 정리가 완료된 후에만 다음 단계로 진행되도록 올바른 경로에 따라 구현할 수 있었다.